

E-BOOKSTORE CAPSTONE

Design, Build & Validation

A full-stack bookstore built with Spring Boot, PostgreSQL, React + TypeScript, and an agentic development workflow using IBM Bob.

Java 21

Spring Boot

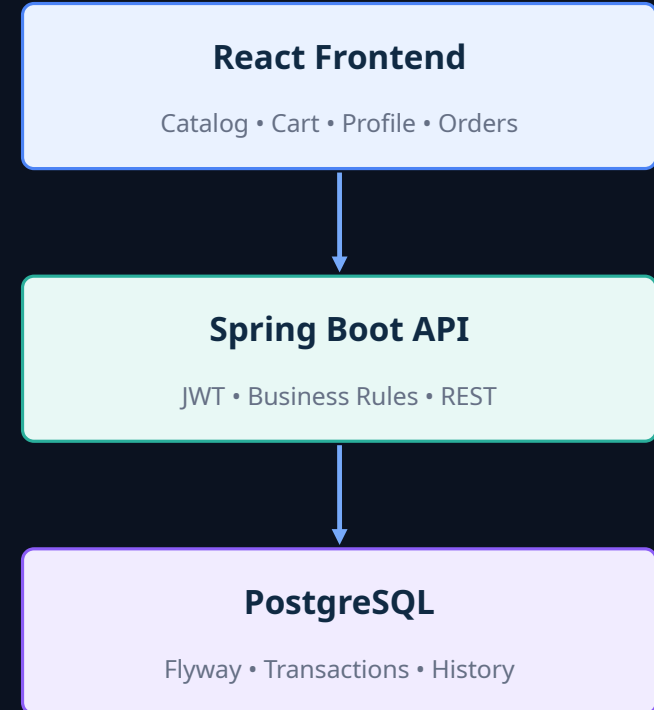
PostgreSQL

React + TS

Presented by

Bhargav Bhutwala

Manager & SME Review



What problem are we solving?

A bookstore customer journey needs to be simple for users and reliable for the business.

Customer pain points

- Hard to discover books across categories, publishers and search.
- Cart and checkout must preserve accurate totals and availability.
- Customers need reusable addresses, order history and clear payment status.
- Times, prices and availability must be understandable in the local context.

Business / system risks

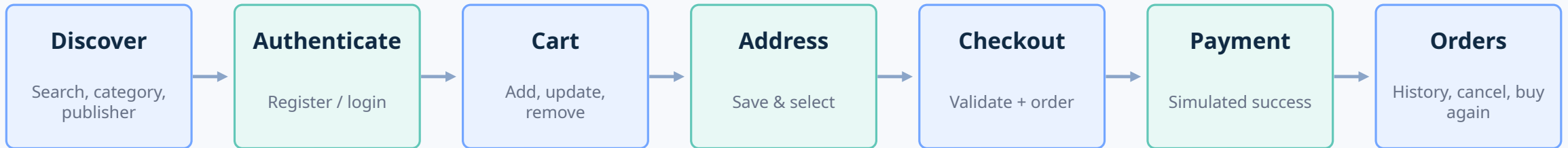
- Overselling when stock validation and inventory updates are inconsistent.
- Price drift if historical order values are not snapshotted.
- Authorization mistakes exposing another customer's data.
- Schema drift when application code, database and API contract evolve separately.

Solution value

- One secure end-to-end purchase journey.
- Transactional checkout with inventory control.
- Design-first API and schema governance.
- Automated unit, integration and end-to-end validation.

MVP scope and customer journey

The MVP focuses on the purchase lifecycle, from discovery to post-purchase actions.



Delivered in the MVP

JWT authentication, catalog/search, related products, cart, address CRUD, order creation, simulated payment, order history, Buy Again, 48-hour cancellation, profile, INR display, IST presentation and book covers.

Intentionally outside MVP

Gift points, coupons, shipments, returns and refunds are documented as future scope rather than partially implemented features.

How I built the solution

A design-first, contract-driven workflow with IBM Bob used as an implementation accelerator.

1

Requirements

Functional scope, business rules, error behavior

2

Data model

Tables, keys, constraints, money and lifecycle fields

3

OpenAPI

REST contract, DTO shapes, auth and errors

4

Backend MVP

14 implementation tasks in a modular monolith

5

Frontend MVP

React milestones aligned to the same API contract

6

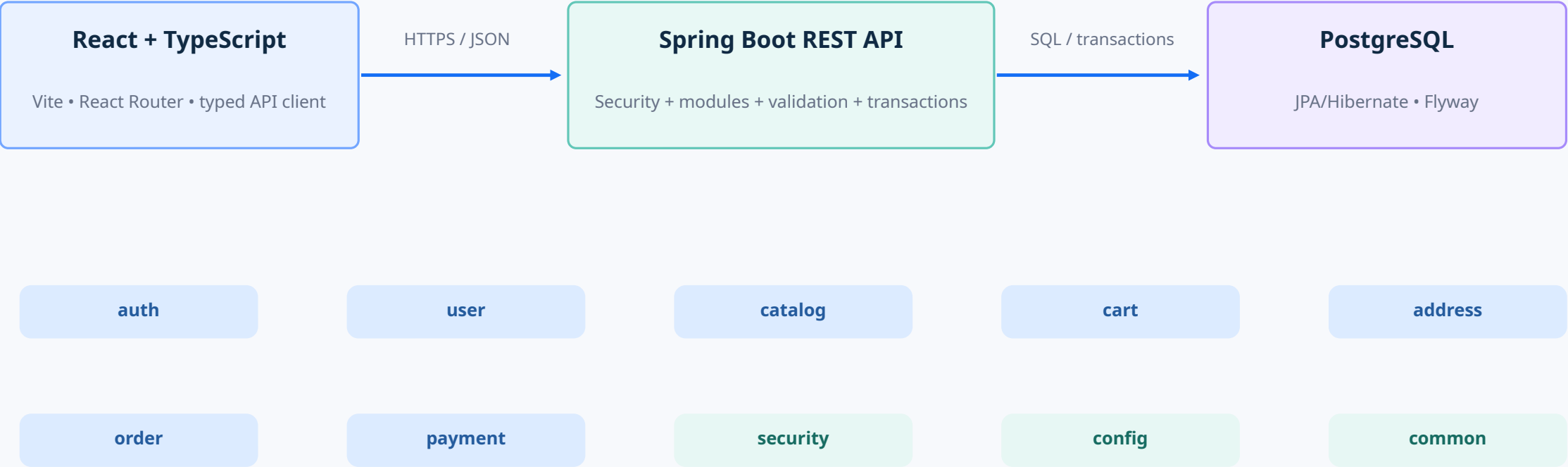
Validate & polish

Tests, E2E, INR, IST, profile, covers, seed data

Bob's role: implement bounded tasks. My role: define the contract, review changes, verify behavior, control scope and preserve a clean Git history.

Solution architecture

A modular monolith keeps the MVP simple while preserving clear domain boundaries.



Why modular monolith? Lower operational complexity for an MVP, but domain packages keep responsibilities separated and future extraction possible.

Backend design and API contract

The backend is organized around domain services, with OpenAPI as the external contract.

Core backend patterns

- Spring MVC REST controllers with DTOs.
- Spring Data JPA repositories and service-layer transactions.
- Centralized validation, authorization and error responses.
- Injected Clock for deterministic time-based rules.
- BigDecimal for monetary values.

Primary API areas

- /auth — register, login, logout
- /users/me — current user
- /categories, /brands, /products — catalog
- /cart, /addresses — customer state
- /orders, /payments — purchase lifecycle

Contract governance

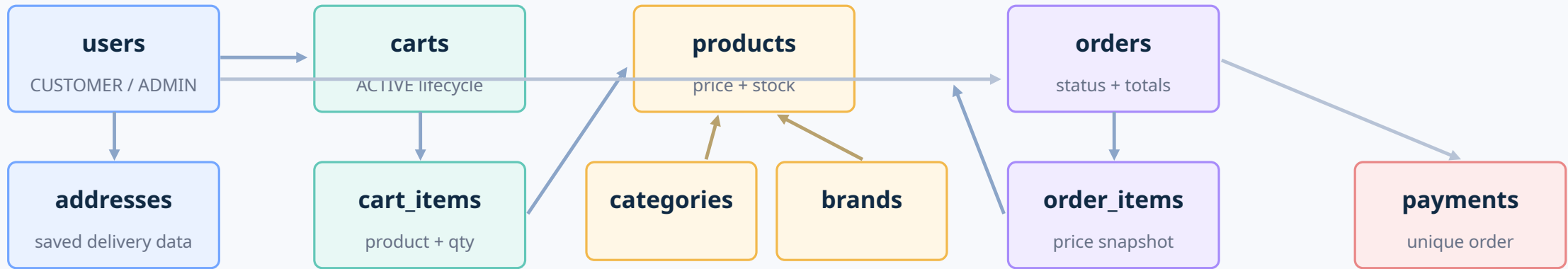
- OpenAPI defines request/response shapes.
- DTOs align to camelCase API fields.
- Database migrations own schema changes.
- Frontend types mirror the contract.
- Phase-2 endpoints remain outside MVP implementation.

Key principle

The database, Java model and TypeScript types do not evolve independently; the contract is changed deliberately and then propagated.

Database design

PostgreSQL provides relational integrity, transactional checkout and durable order history.

**10**

MVP tables

12

Flyway migrations

12,2

NUMERIC precision

48h

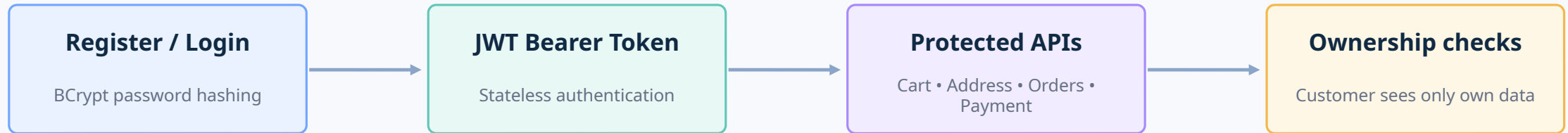
Cancellation window

Why snapshots?

Order items keep purchase-time price even when catalog prices change later.

Security and authorization

Stateless JWT authentication protects customer operations without server-side session state.



Authentication controls

- BCrypt hashes passwords.
- Bearer token required on protected endpoints.
- 401 handling clears invalid frontend session.

Authorization controls

- Roles: CUSTOMER and ADMIN.
- Ownership enforced in service layer.
- Unauthorized resource access follows the designed not-found behavior.

Input & error controls

- Bean validation for request fields.
- Centralized ErrorResponse structure.
- Conflict handling for stock and business rules.

Frontend architecture and UX

React + TypeScript consumes the same API contract and keeps customer state predictable.

Application shell

- React Router protected/public routes
- Header navigation and authenticated profile
- Shared loading, error and empty states
- Vite proxy to Spring Boot API

Feature modules

- auth
- catalog
- cart
- address
- checkout
- payment
- orders
- profile

Shared types & utilities

- OpenAPI-aligned DTO types
- formatCurrency → INR
- formatDateTime → Asia/Kolkata
- Reusable ProductGrid / ProductCard

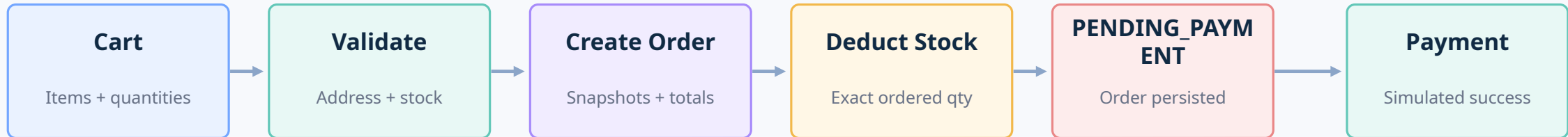
Customer polish

- Book cover image + fallback
- 70-book demo catalog
- Address CRUD inside profile
- Responsive catalog and product pages

Design principle: server remains authoritative for prices, totals, stock and order state; the frontend renders and orchestrates the user journey.

Checkout, inventory and payment flow

The most important business transaction is deliberately split into order creation and payment.



Why deduct at order creation?

- Prevents checkout from reserving stock that still appears available.
- Order creation and inventory change are in one transaction.
- Payment must not decrement stock again.

Failure behavior

- Insufficient stock rejects the order.
- No negative inventory.
- Transactional rollback avoids partial persistence.
- Exact-stock purchase is allowed and results in zero.

Historical accuracy

- Order items snapshot title/price at purchase time.
- Later catalog price changes do not rewrite order history.
- Payment is linked uniquely to its order.

Localization and time correctness

Customer-facing INR and IST were added without corrupting backend data semantics.

Currency: INR

Shared Intl.NumberFormat

Locale: en-IN
Currency: INR
Example: ₹1,099.00
Server still uses BigDecimal /
NUMERIC(12,2)

Time: real instant → IST

- Backend uses OffsetDateTime with injected UTC Clock.
- JSON includes an offset such as +00:00.
- Frontend parses the instant and formats with Asia/Kolkata.
- No manual +05:30 arithmetic.

Why the fix mattered

- LocalDateTime serialized without offset was ambiguous in JavaScript.
- Offset-aware timestamps preserve the real instant.
- placedAt, paidAt and cancellationDeadline now display correctly in IST.

UTC in the backend + IST in the UI = correct instant, correct local presentation.

Quality engineering and test strategy

Tests validate service logic, real PostgreSQL behavior and end-to-end user journeys.

282/282

Backend tests passed

115/115

Frontend unit/component

14

Playwright scenarios

59/59

Targeted stock/time tests

Backend coverage

- JUnit 5 + Mockito service tests
- Controller / integration tests
- PostgreSQL Testcontainers
- Security, validation, cancellation, stock, timestamps

Frontend coverage

- Vitest + React Testing Library
- Catalog, cart, payment, orders, profile
- Currency/date-time formatters
- Protected route behavior

E2E learning

- Real backend + Vite frontend
- Covers auth, catalog, cart, checkout/payment, orders
- Shared DB state can make parallel runs flaky
- Seed reset / serial execution improves determinism

Problems encountered and how they were solved

The project improved through targeted debugging rather than broad rewrites.

Issue	Root cause	Resolution
Book covers	Original schema had no image field	V12 nullable image_url + API/DTO/type propagation + fallback UI
Currency	USD-style demo prices and \$ formatting	INR seed values + shared Intl.NumberFormat(en-IN, INR)
Time	LocalDateTime lost UTC context in JSON	OffsetDateTime + UTC Clock + Asia/Kolkata UI formatting
Stock verification	User observed unchanged stock	Confirmed transactional decrement; stale/reseeded data explained the observation
Legacy seed data	Old hyphenated ISBN rows caused duplicates	Deactivated legacy catalog rows; preserved historical references
E2E stability	Parallel tests share real DB and mutate stock	Reset deterministic seed / run serially for stable full-stack verification

Key technical decisions and trade-offs

The architecture favors correctness, maintainability and MVP delivery speed.



Modular monolith

Chosen over microservices

Less deployment complexity;
domain boundaries remain clear



Flyway

Chosen over Hibernate DDL

Repeatable schema history and
deployment control



JWT + BCrypt

Stateless auth

Simple API scaling; frontend handles
token lifecycle



BigDecimal

Money representation

Avoids binary floating-point errors



Order-time stock decrement

Inventory policy

Prevents overselling; payment
cannot double-decrement



Offset-aware timestamps

Time correctness

Real instant preserved; local
timezone becomes presentation
concern



Testcontainers

Integration fidelity

Tests real PostgreSQL behavior
instead of an in-memory substitute



OpenAPI-first

Contract governance

Keeps backend DTOs and frontend
types aligned

Current result: a complete MVP purchase journey

The application is demo-ready and has been manually verified end to end.



Catalog

70 seeded books, covers, search/filter, related products



Localization

INR prices and IST customer-facing timestamps



Profile

User details + reusable address CRUD



Checkout

Stock validation + transactional order creation



Payment

Separate simulated payment, no double stock decrement



Orders

History, details, Buy Again and 48-hour cancellation

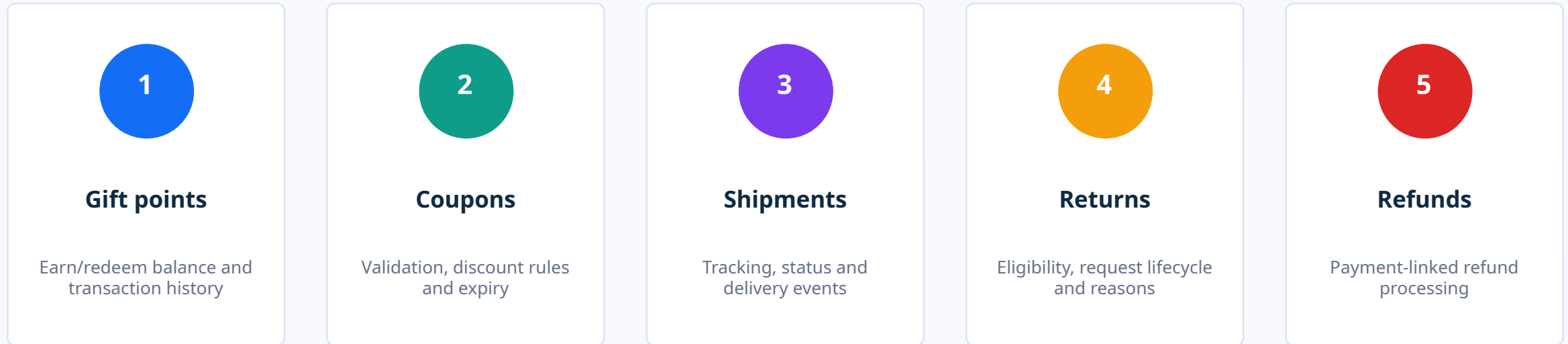
Suggested live demo sequence

Login → Search a book → Product detail → Cart qty 2 → Profile/address → Checkout → Payment → Orders → Verify stock in PostgreSQL

Keep the live demo to 4-5 minutes and let the slides explain the architecture around it.

Future scope / Phase 2

The next iteration can extend the purchase lifecycle without destabilizing the MVP core.



How I would approach Phase 2

Keep the same contract-first workflow: update requirements → data model → OpenAPI → migrations → backend → frontend → focused regression tests.
Avoid combining multiple Phase-2 domains into one uncontrolled agent task.

SME discussion topics to be ready for

Expect “why did you choose this?” questions more than “what file contains this?”.

Architecture	Why modular monolith? When would you move to microservices?	Database	Why Flyway? How do transactions prevent partial checkout?
Inventory	Why decrement stock at order creation instead of payment?	Security	How do JWT, roles and ownership checks work together?
Money	Why BigDecimal / NUMERIC instead of double?	Time	Why UTC/OffsetDateTime backend and IST only in presentation?
API	How does OpenAPI keep frontend and backend aligned?	Testing	What is tested with Mockito vs Testcontainers vs Playwright?
Agentic workflow	How did you prevent Bob from changing scope or architecture?	Scalability	What would you change for high traffic, real payments or distributed inventory?

Conclusion

A working, tested MVP built with clear contracts, controlled agentic development and end-to-end validation.

What was delivered

- Full customer purchase journey
- Secure backend + responsive frontend
- PostgreSQL schema and migrations
- Automated validation across layers

What I demonstrated

- Design-first engineering
- Transactional business logic
- API/database/frontend alignment
- Root-cause debugging and test-driven fixes

Submission package

- GitHub source repository
- README + design documents
- Demo seed script
- Recorded demo video / GitHub Release
- This presentation

Thank you — Questions?

Recommended presentation: 12–15 minutes + 5 minutes for SME Q&A